BLG 422E: Computer Networks - Homework
Taha Temiz 150210055

Q1) Stop-and-Wait & Go-Back-N Protocols in OMNET++

Setup and Implementation

I implemented reliable data transfer protocols using the Rdt3Network topology and RdtPacket message structure provided in the homework. The system consists of a client that requests a file and a server that transmits it in segments. The protocol operates as Stop-and-Wait when windowSize=1 and as Go-Back-N when windowSize>1.

The file being transferred is 200,000 bytes, divided into 200 segments of 1,000 bytes each. The client sends cumulative ACKs for received packets, and the server maintains a sliding window for Go-Back-N operation. When a timeout occurs, Stop-and-Wait retransmits only the unacknowledged packet, while Go-Back-N retransmits all packets in the current window starting from the first unacknowledged one.

I tested the protocols under three channel conditions (PER = 0.0, 0.1, 0.2) with three window sizes (1, 10, 100). The key metrics I collected were completion time, throughput, average one-way delay, and total retransmissions.

Results and Discussion

A) PER = 0.0 (No Packet Loss)

With perfect channel conditions (PER = 0.0), there were no packet losses and consequently zero retransmissions across all window sizes. This served as an effective baseline to verify my implementation. I see the effect of the window size on performance.

windowSize = 1 (Stop-and-Wait):

- Completion time: 4.20 seconds
- Throughput: 0.381 Mbps
- Retransmissions: 0

This was very slow. Because the server sends only one packet and then waits for its ACK before sending the next, the protocol spends most of its time idle. With a 20ms round-trip time (10ms delay each way), the server can only send about 50 packets per second, which explains why it took over 4 seconds to transfer just 200KB.

windowSize = 10:

- Completion time: 0.446 seconds
- Throughput: 3.59 Mbps
- Retransmissions: 0

This is where Go-Back-N really shows its advantage. By allowing up to 10 packets in flight simultaneously, the protocol achieves roughly 9.4x better throughput compared to Stop-and-Wait. The completion time dropped from 4.2 seconds to less than half a second. The server can keep the channel busy instead of waiting idle for ACKs.

windowSize = 100:

- Completion time: 0.210 seconds
- Throughput: 7.60 Mbps
- Retransmissions: 0

Doubling the window size again gave another significant boost - completion time halved once more. At this point, the protocol is approaching the theoretical maximum throughput of the 10 Mbps channel. The ability to have 100 packets in flight means the pipeline stays full, minimizing idle time.

B) PER = 0.1 (Moderate Packet Loss)

When I introduced 10% packet loss, performance degraded significantly across all configurations. However, the degree of degradation varied dramatically based on window size, revealing an important trade-off in Go-Back-N.

windowSize = 1 (Stop-and-Wait):

- Completion time: 13.40 seconds
- Throughput: 0.119 Mbps
- Retransmissions: 46 (23% of 200 packets)

Stop-and-Wait became about 3.2x slower than the lossless case. With 10% loss, roughly every 10th packet gets corrupted and needs retransmission. Since the protocol waits for an ACK after each packet, a single loss adds one full timeout period (plus retransmission time). The 46 retransmissions meant 46 timeouts, each adding about 200ms, which explains the significant delay.

Interestingly, the retransmission percentage (23%) is more than double the PER (10%). This happens because retransmitted packets can also be lost, requiring another retransmission. Some packets might even be lost multiple times before successfully getting through.

windowSize = 10:

- Completion time: 5.36 seconds
- Throughput: 0.299 Mbps
- Retransmissions: 213 (106.5%)

Here's where Go-Back-N's weakness starts to show. The completion time is still much better than Stop-and-Wait, but the retransmission count exploded to 213. This is the classic

Go-Back-N penalty: when packet #5 gets lost, the receiver keeps receiving packets #6, #7, #8, #9 but has to discard them all because they're out of order. When the timeout fires, the server retransmits everything from #5 onwards, even though #6-#9 were actually received correctly the first time.

The retransmission ratio being 106.5% means that on average, every packet was transmitted at least twice. Some packets were probably sent three or four times due to multiple losses.

windowSize = 100:

- Completion time: 4.99 seconds
- Throughput: 0.321 Mbps
- Retransmissions: 1773 (886.5%)

This result really surprised me. The completion time barely improved compared to windowSize=10 (only 0.37 seconds faster), but the retransmissions increased to 1773. The large window made every single packet loss extremely expensive. When one packet in a window of 100 gets lost, all subsequent packets get retransmitted even if they were received correctly.

This shows that bigger isn't always better. The extra throughput from the larger window is almost completely negated by the massive amount of wasted retransmissions. The channel ends up congested with redundant data, and the server is constantly timing out and restarting from way back in the sequence.

C) PER = 0.2 (High Packet Loss)

At 20% packet loss, the situation became even more dramatic. The Go-Back-N window size that worked best shifted downward, and windowSize=100 actually performed worse than windowSize=10 in terms of completion time.

windowSize = 1 (Stop-and-Wait):

- Completion time: 22.80 seconds
- Throughput: 0.070 Mbps
- Retransmissions: 93 (46.5%)

Stop-and-Wait took over 22 seconds - more than 5 times slower than the lossless case. With 20% loss, roughly every 5th packet failed. The protocol spent so much time in timeout/retransmission cycles that throughput dropped to just 70 Kbps. However, it remained relatively stable because each loss only affects one packet at a time.

windowSize = 10:

- Completion time: 10.21 seconds
- Throughput: 0.157 Mbps

- Retransmissions: 449 (224.5%)

Go-Back-N with windowSize=10 performed the best among all three configurations at PER=0.2. While the retransmission count was still high (over twice the number of packets), the completion time was significantly better than both Stop-and-Wait and the larger window. This is the sweet spot - enough parallelism to maintain decent throughput, but not so large that every loss triggers an avalanche of unnecessary retransmissions.

windowSize = 100:

- Completion time: 11.63 seconds
- Throughput: 0.138 Mbps
- Retransmissions: 4337 (2168.5%)

This was eye-opening. WindowSize=100 actually took longer to complete than windowSize=10, despite having 10x more parallelism. The problem is the 4,337 retransmissions. The protocol spent most of its time retransmitting old packets rather than making forward progress.

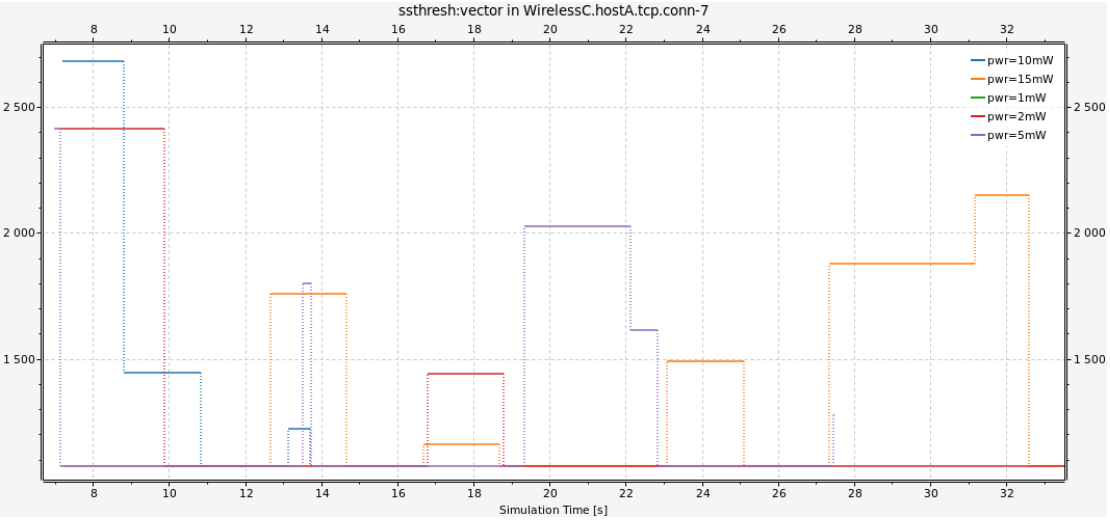
The throughput (0.138 Mbps) was also lower than windowSize=10, confirming that the huge window backfired. This definitively answers the homework question: No, the highest N does not always make sense for Go-Back-N. At high loss rates, a moderate window size is much more efficient.

Q2) TCP Performance in Dynamic Networks

For this part, I tried to configure a TCP client/server by extending Wireless13 in the tutorials of Inet Framework. I replaced the UDP application in the Wireless13 with a TCP client/server pair. The simulations were conducted using the TCP NewReno congestion control algorithm. I observed the metrics while changing transmission power. The objective was to analyze TCP behavior across transport, network, MAC, and physical layers and to evaluate how wireless dynamics affect performance.
A basic TCP client app was deployed on hostA, and a generic TSP server app was deployed on hostB. I limited the queue capacity to 10 packets to create a realistic bottleneck at the sender side. Transmission power parameters are 1mW, 2mW, 5mW, 10mW, 15mW.

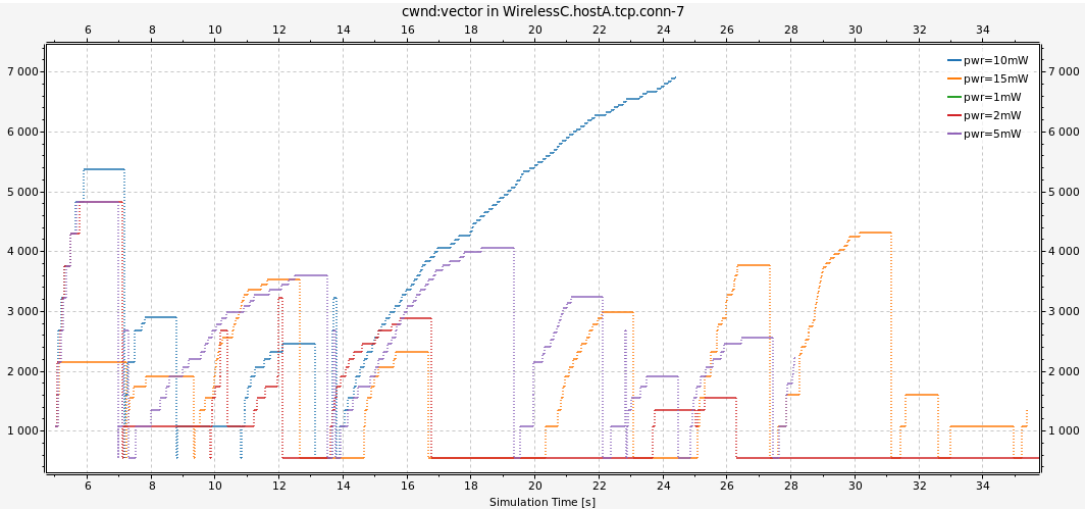
- **TCP State Variables**

  ssthresh



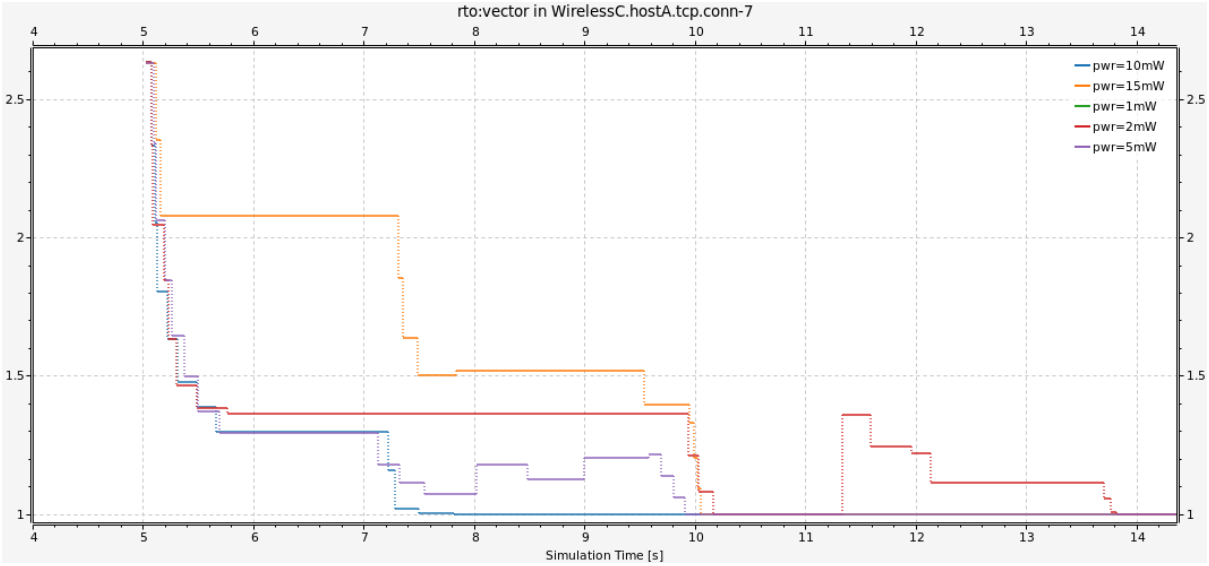
This chart shows the evolution of the TCP slow start threshold (ssthresh) for different transmission power levels. The threshold remains constant between loss events and is updated only when TCP detects congestion, resulting in piecewise constant behavior. At higher transmission power (10 mW), ssthresh stabilizes quickly due to fewer packet losses and stable connectivity. In contrast, at lower power levels, frequent losses and route instabilities cause repeated updates to ssthresh. This situation claims TCP's continuous adaptation to changing network conditions.

cwnd



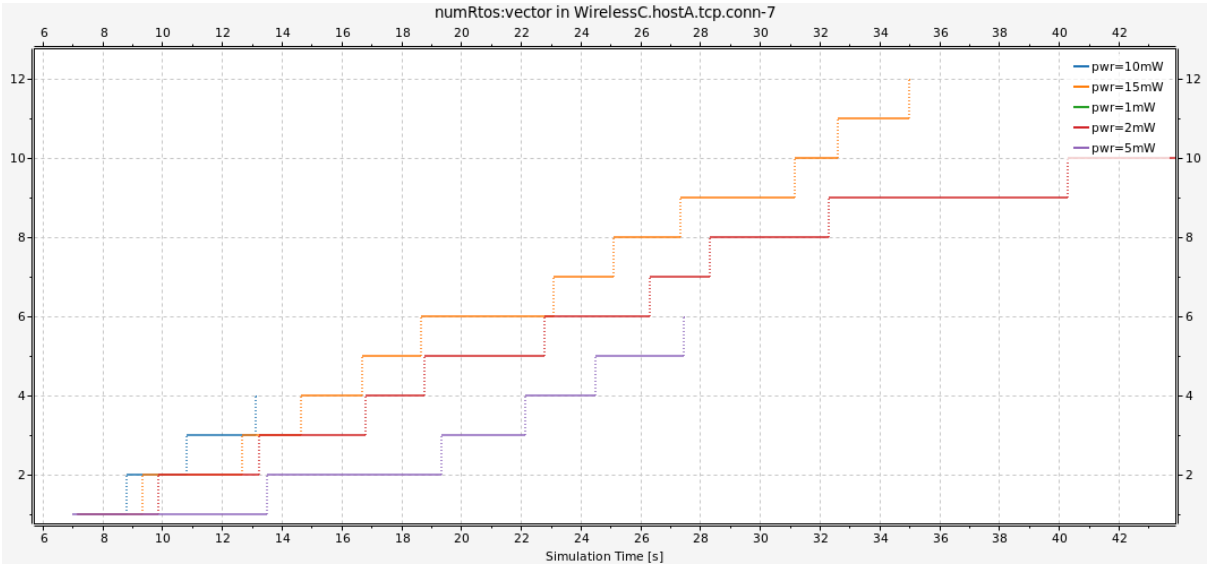
This chart shows the TCP congestion window evolution at hostA for different transmission power levels. Higher power levels provide more stable connectivity, allowing TCP to maintain larger congestion windows with fewer resets. At lower power levels, frequent packet losses and route failures trigger repeated timeouts, causing cwnd to collapse and re-enter slow start.

RTO



This chart shows the behaviour of the TCP retransmission timeout (RTO) under different transmission power levels. At high power (10 mW), RTO quickly converges to a low and stable value, indicating consistent RTT measurements and reliable connectivity. In contrast, at lower power levels, RTO remains elevated or exhibits frequent increases due to high RTT variance and packet losses caused by wireless instability and route changes. Elevated RTO values lead to longer timeout periods, explaining the prolonged congestion window collapses and reduced throughput observed in low-power scenarios

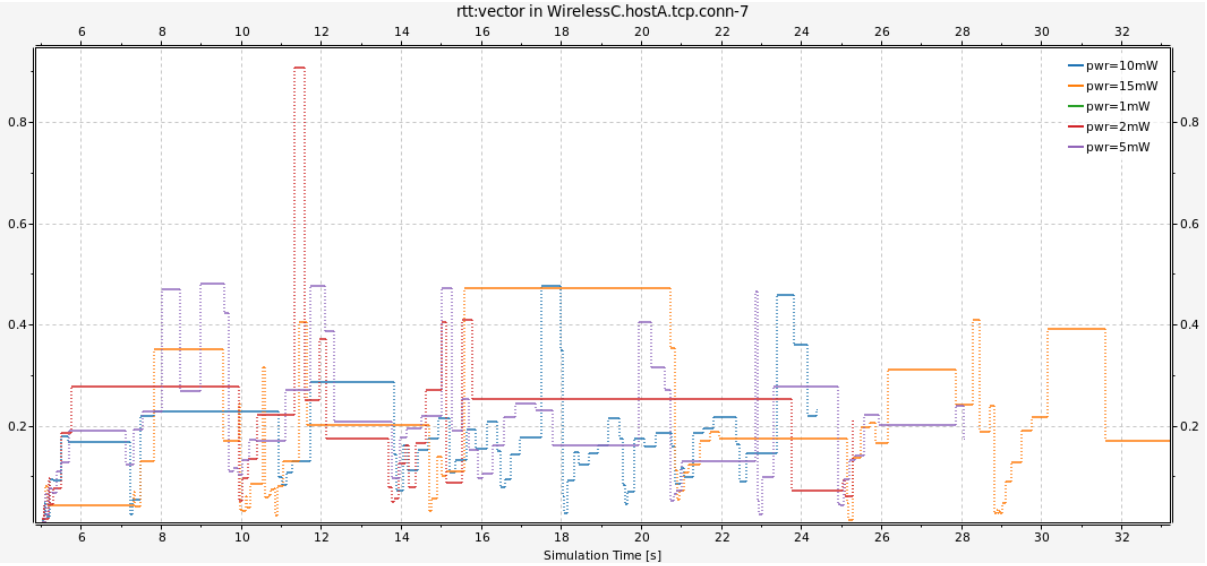
Number of Retransmission Timeouts



This chart presents the cumulative number of TCP retransmission timeouts (numRtos) over time for different transmission power levels. At high power (10 mW), only a few timeout events occur during the initial connection phase. In contrast, at lower power levels, timeouts accumulate steadily throughout the simulation. The high

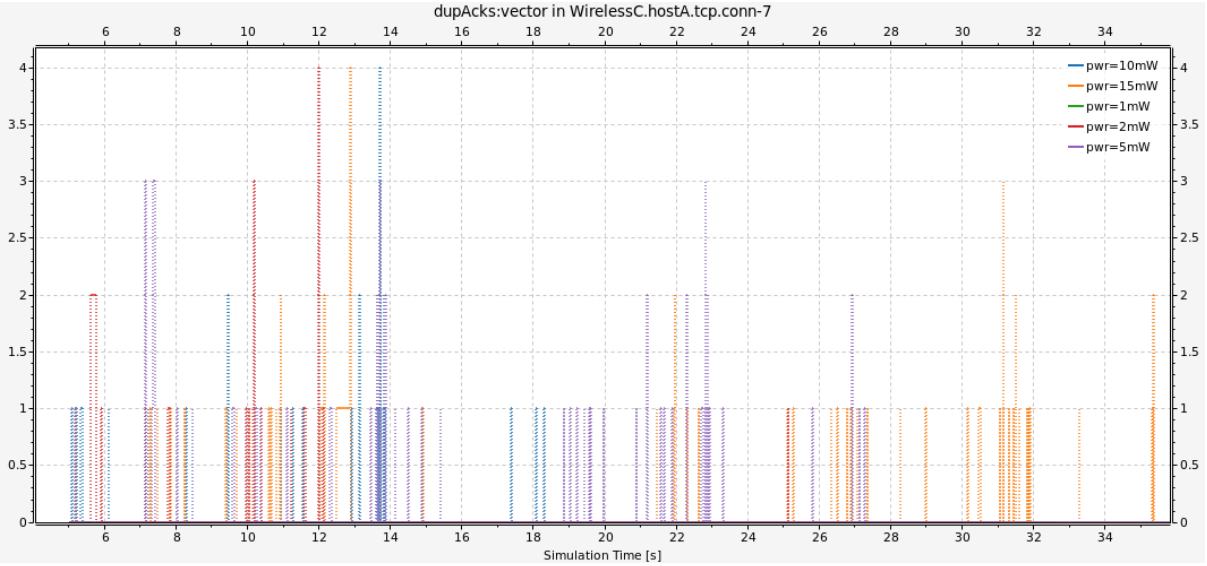
number of timeout-based retransmissions explains the repeated congestion window collapses and low throughput observed in low-power scenarios.

RTT



This chart shows the changing of the TCP round-trip time (RTT) for different transmission power levels. At high power (10 mW), RTT remains relatively low and stable, indicating reliable links and consistent ACK reception. As transmission power decreases, RTT exhibits higher variance and frequent spikes caused by MAC-layer retransmissions, route changes, and increased queueing delays. This increased RTT variability leads to inflated retransmission timeouts and frequent timeout events, explaining the degraded TCP performance observed in low-power scenarios.

Duplicated ACK Bursts

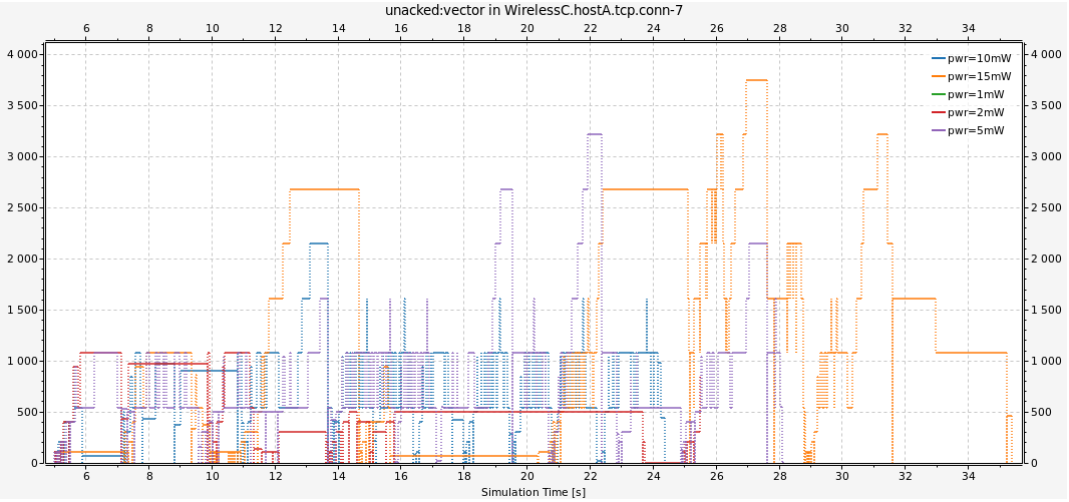


This chart points to the occurrence of duplicate ACK (dupACK) bursts at the TCP sender for different transmission power levels. At lower transmission power levels, dupACK bursts occur more frequently and are clustered in time. These dupACK bursts trigger fast retransmit and fast recovery mechanisms in TCP NewReno. In contrast, at higher transmission power levels, dupACK activity is sparse, reflecting more reliable packet delivery and stable network conditions

MSS

I configured the MSS value of the TCP pair as 536 bytes. Minimum IPv4 MTU is 576 bytes. IP header is 20 bytes, TCP header is 20 bytes. So 576-40 = 536 bytes to avoid fragmentation.

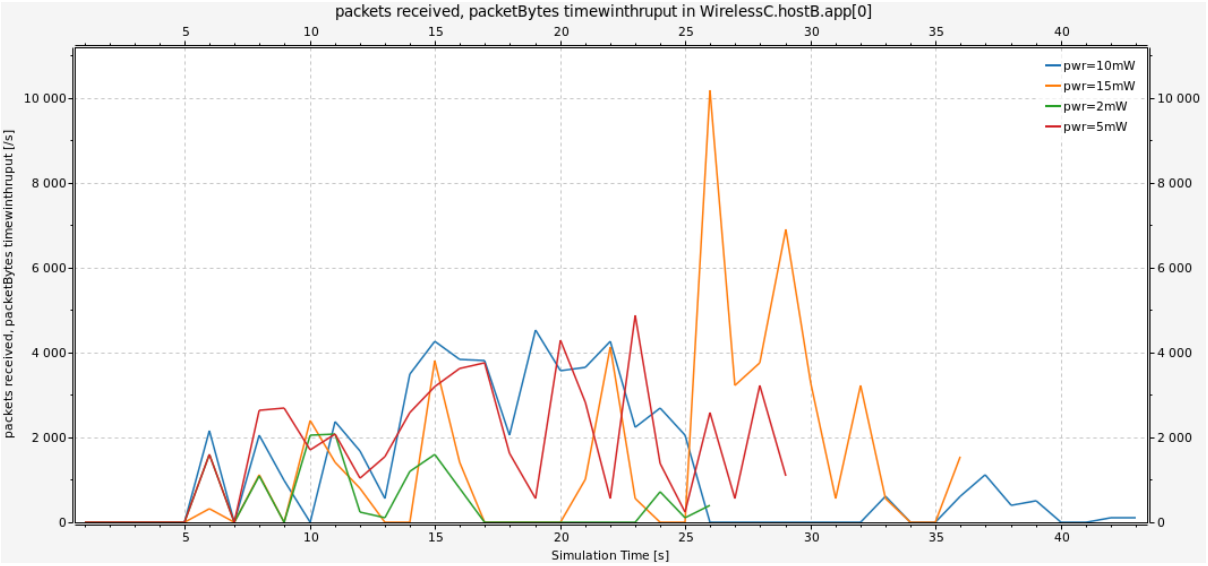
Flight Size



This chart depicts the evolution of the TCP flight size, measured as the number of unacknowledged bytes at the sender. At high transmission power, TCP maintains a stable flight size that indicates an effective pipeline. At medium and low power levels, the flight size exhibits large oscillations and frequent collapses due to packet losses and timeout-based retransmissions. These frequent pipeline drains significantly reduce throughput and explain the performance degradation observed under unstable wireless conditions**.**

- **Performance metrics**

Throughput



Increasing transmission power improves TCP throughput by reducing packet loss, stabilizing congestion window growth, and maintaining a consistent flight size. At low power levels, frequent disconnections and timeouts severely limit throughput. While high power occasionally produces large instantaneous throughput peaks, overall performance is best characterized by stability rather than peak values.

Delivery Ratio

Experiment	Measurement	Replica	Module	Module Display Pat	Name	Value
Q2_PowerTest	$pwr=10mW	#0	WirelessC.hostA.app[	WirelessC.hostA.app[	packetSent:sum	53 400
Q2_PowerTest	$pwr=15mW	#0	WirelessC.hostA.app[	WirelessC.hostA.app[	packetSent:sum	50 000
Q2_PowerTest	$pwr=1mW	#0	WirelessC.hostA.app[	WirelessC.hostA.app[	packetSent:sum	0
Q2_PowerTest	$pwr=2mW	#0	WirelessC.hostA.app[	WirelessC.hostA.app[	packetSent:sum	13 400
Q2_PowerTest	$pwr=5mW	#0	WirelessC.hostA.app[	WirelessC.hostA.app[	packetSent:sum	50 000

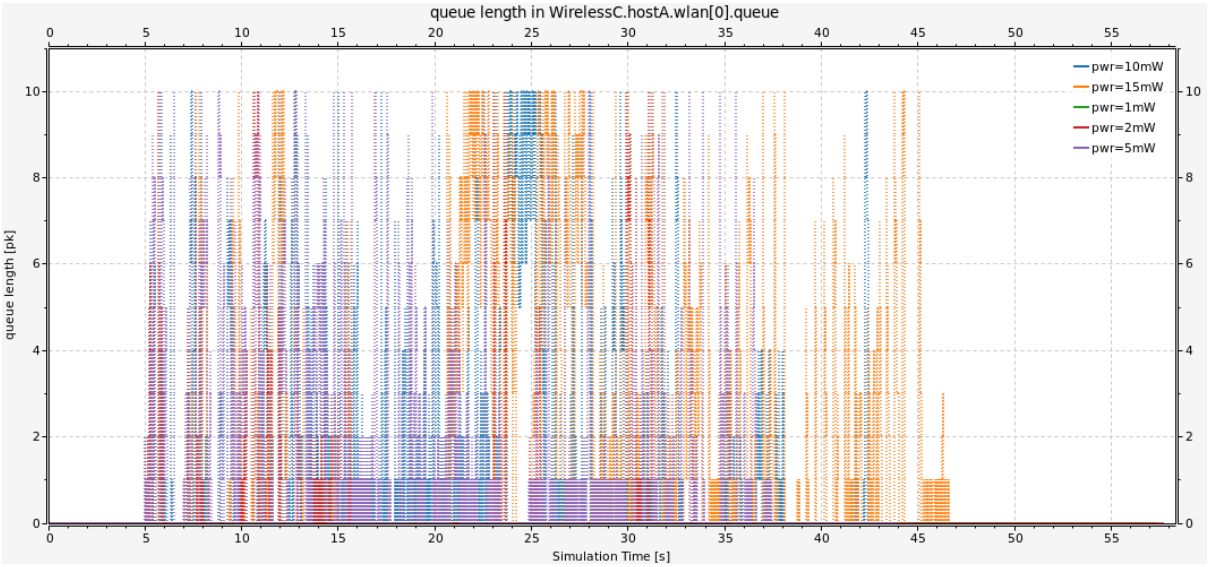
packetSent:sum(packetBytes) by hostA

Experiment	Measurement	Replica	Module	Module Display Pat	Name	Value
Q2_PowerTest	$pwr=10mW	#0	WirelessC.hostB.app[	WirelessC.hostB.app[	packetReceived	53 400
Q2_PowerTest	$pwr=15mW	#0	WirelessC.hostB.app[	WirelessC.hostB.app[	packetReceived	50 000
Q2_PowerTest	$pwr=1mW	#0	WirelessC.hostB.app[	WirelessC.hostB.app[	packetReceived	0
Q2_PowerTest	$pwr=2mW	#0	WirelessC.hostB.app[	WirelessC.hostB.app[	packetReceived	11 900
Q2_PowerTest	$pwr=5mW	#0	WirelessC.hostB.app[	WirelessC.hostB.app[	packetReceived	50 000

packetReceived:sum(packetBytes) by hostB
So the delivery ratio is for:
1mW -> Zero because no packet is sent.
2mW-> 11900/13400 = 0.89
5mW-> 50000/50000 =1
10mW-> 1
15mW-> 1

- **Network behavior**

queueLength



This chart shows the evolution of the WLAN MAC queue length at the TCP sender. At high transmission power, the queue frequently approaches its maximum capacity, indicating that the bottleneck shifts to the MAC layer as TCP injects data aggressively. At medium power levels, queue occupancy fluctuates significantly due to repeated congestion control reactions triggered by packet losses. At low transmission power, the queue remains mostly empty, showing that performance is limited by link unreliability rather than buffer congestion.

Packet loss

Packet loss was evaluated using both TCP retransmission statistics and application-layer delivery ratios. As transmission power decreases, packet loss increases significantly due to wireless channel impairments and frequent route disruptions. This leads to a higher number of TCP retransmissions and timeout events, which in turn cause repeated congestion window reductions and throughput degradation. At higher transmission power levels, packet loss is reduced, resulting in fewer retransmissions and more stable TCP performance. So, for this metric I can not show a specific chart.

- **PHY/MAC layer**

PHY data rate

Experiment	Measurement	Replica	Module	Module Display Pat	Name	Value
Q2_PowerTest	$pwr=10mW	#0	WirelessC.hostA.wlar	WirelessC.hostA.wlar	bitrate	1e+06bps
Q2_PowerTest	$pwr=15mW	#0	WirelessC.hostA.wlar	WirelessC.hostA.wlar	bitrate	1e+06bps
Q2_PowerTest	$pwr=1mW	#0	WirelessC.hostA.wlar	WirelessC.hostA.wlar	bitrate	1e+06bps
Q2_PowerTest	$pwr=2mW	#0	WirelessC.hostA.wlar	WirelessC.hostA.wlar	bitrate	1e+06bps
Q2_PowerTest	$pwr=5mW	#0	WirelessC.hostA.wlar	WirelessC.hostA.wlar	bitrate	1e+06bps

The bitrate value is 1e+06bps and it is constant.

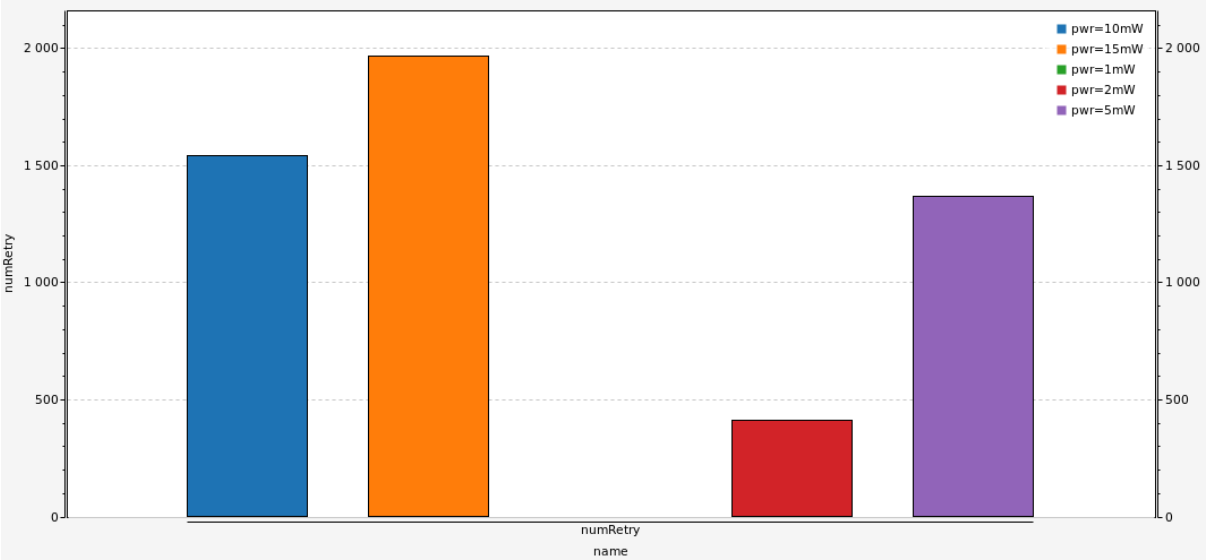
Experiment	Measurement	Replica	Module	Module Display Pat	Name	Value
Q2_PowerTest	$pwr=10mW	#0	WirelessC.hostA.wlar	WirelessC.hostA.wlar	bandwidth	2e+06Hz
Q2_PowerTest	$pwr=15mW	#0	WirelessC.hostA.wlar	WirelessC.hostA.wlar	bandwidth	2e+06Hz
Q2_PowerTest	$pwr=1mW	#0	WirelessC.hostA.wlar	WirelessC.hostA.wlar	bandwidth	2e+06Hz
Q2_PowerTest	$pwr=2mW	#0	WirelessC.hostA.wlar	WirelessC.hostA.wlar	bandwidth	2e+06Hz
Q2_PowerTest	$pwr=5mW	#0	WirelessC.hostA.wlar	WirelessC.hostA.wlar	bandwidth	2e+06Hz

The bandwidth value is 2e+06Hz and it is constant.

Effective MAC Rate

The effective MAC-layer data rate was evaluated indirectly using the achieved TCP throughput, which captures the combined effects of contention, retransmissions, and wireless losses.

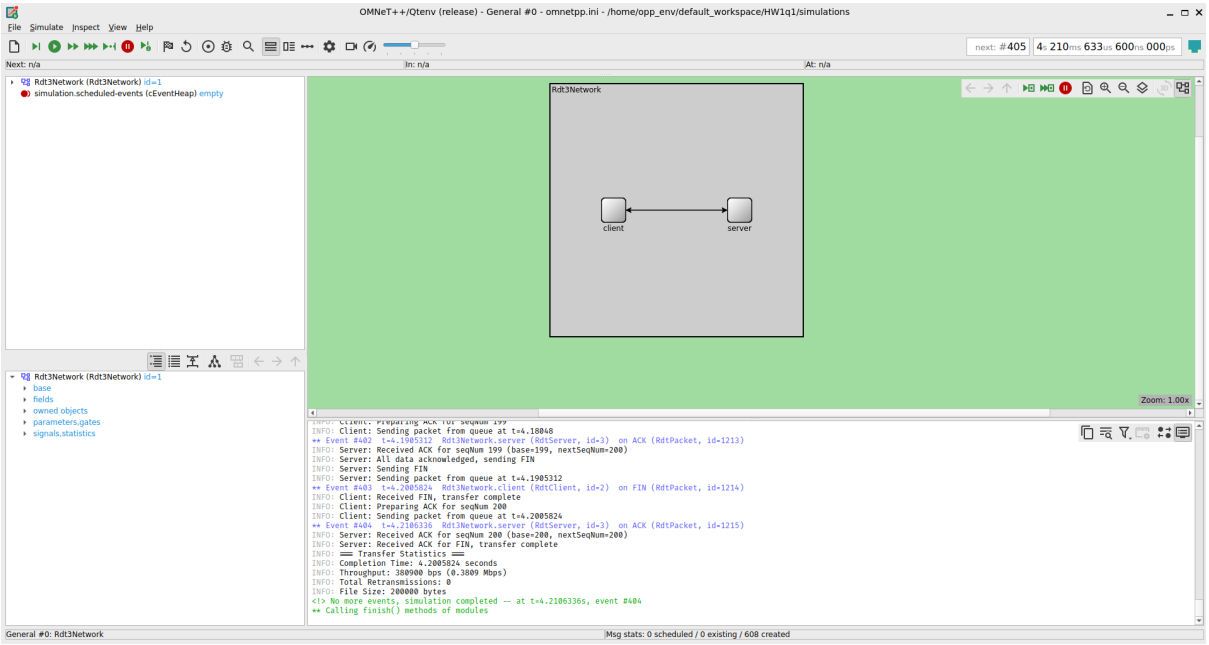
MAC retransmissions



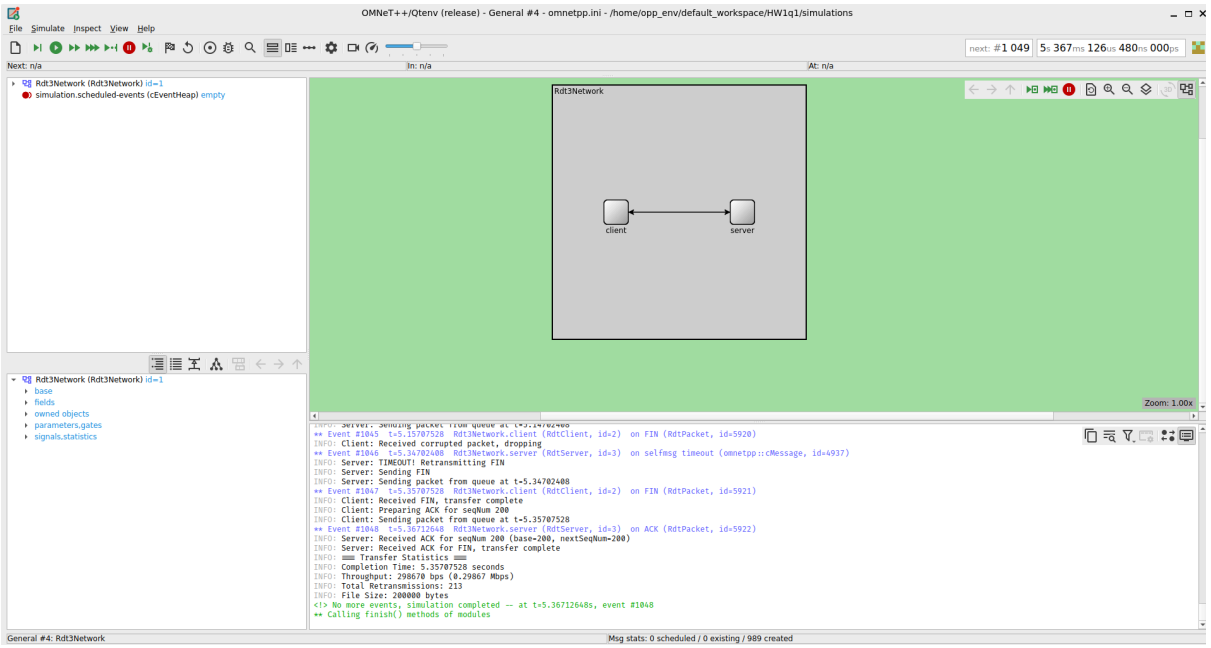
This histogram presents the total number of MAC-layer retransmissions measured at the WLAN interface of the TCP sender. At very low transmission power, the retry count remains low due to limited data injection caused by frequent TCP timeouts. As transmission power increases, TCP injects more traffic into the network, leading to increased MAC activity and a higher number of retransmissions. At the highest transmission power level, the retry count reaches its maximum due to increased channel contention and collisions, despite improved signal quality.

Appendix

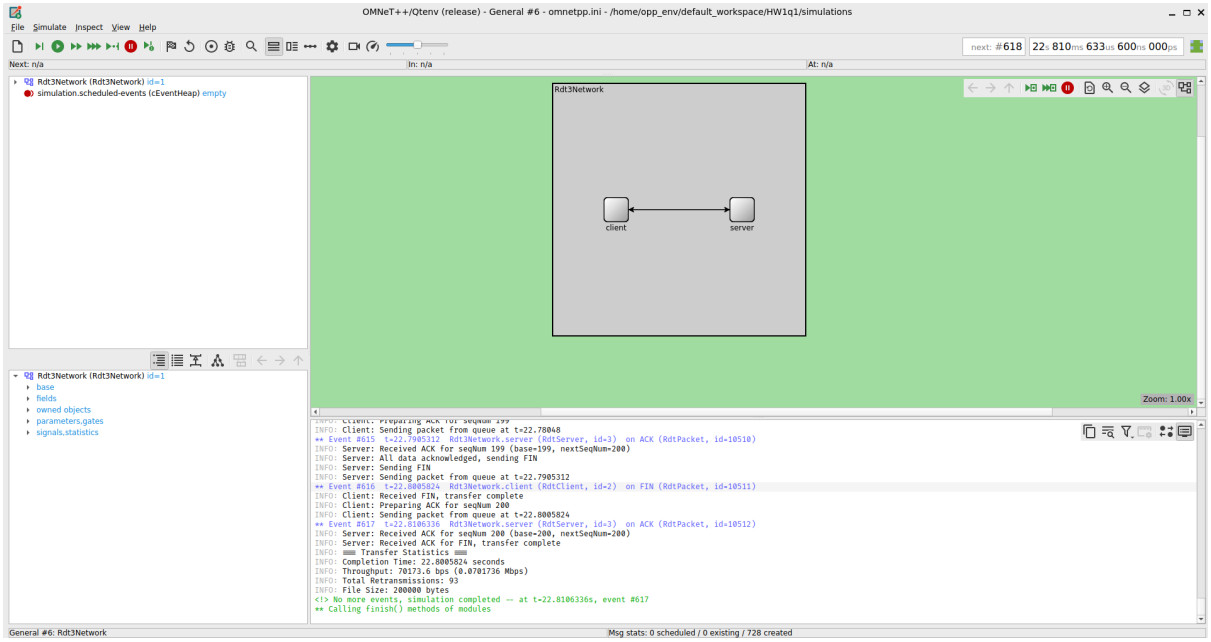
Screenshots for Q1



chanelper 0.0 windowsize 1

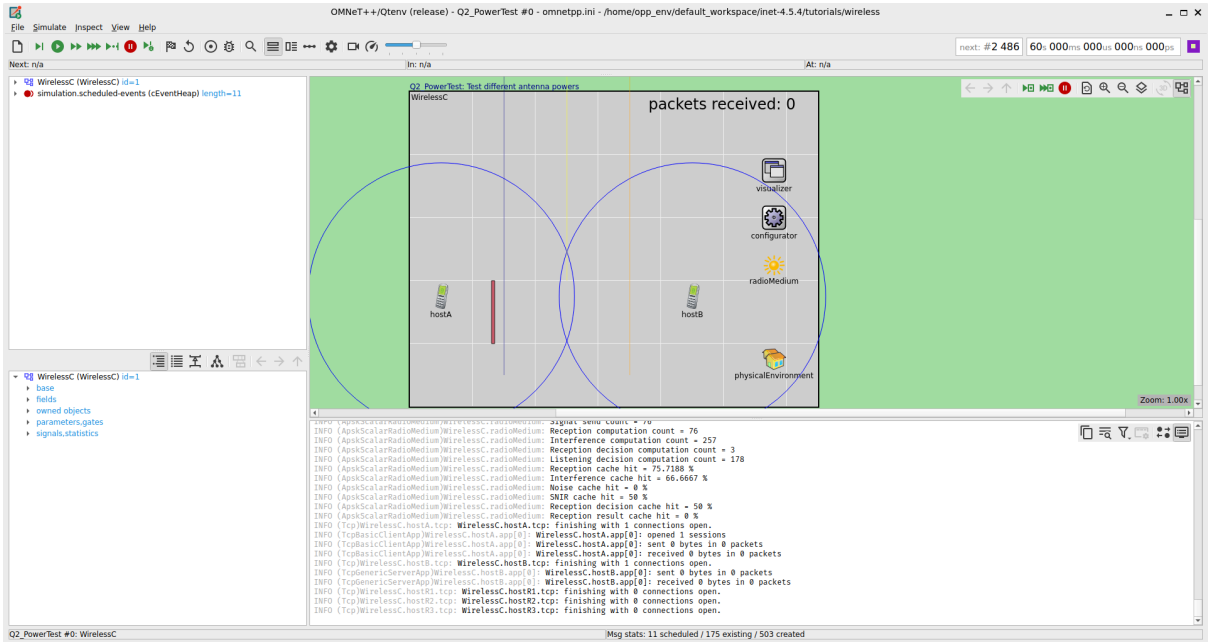


chanelper 0.1 windowsize10

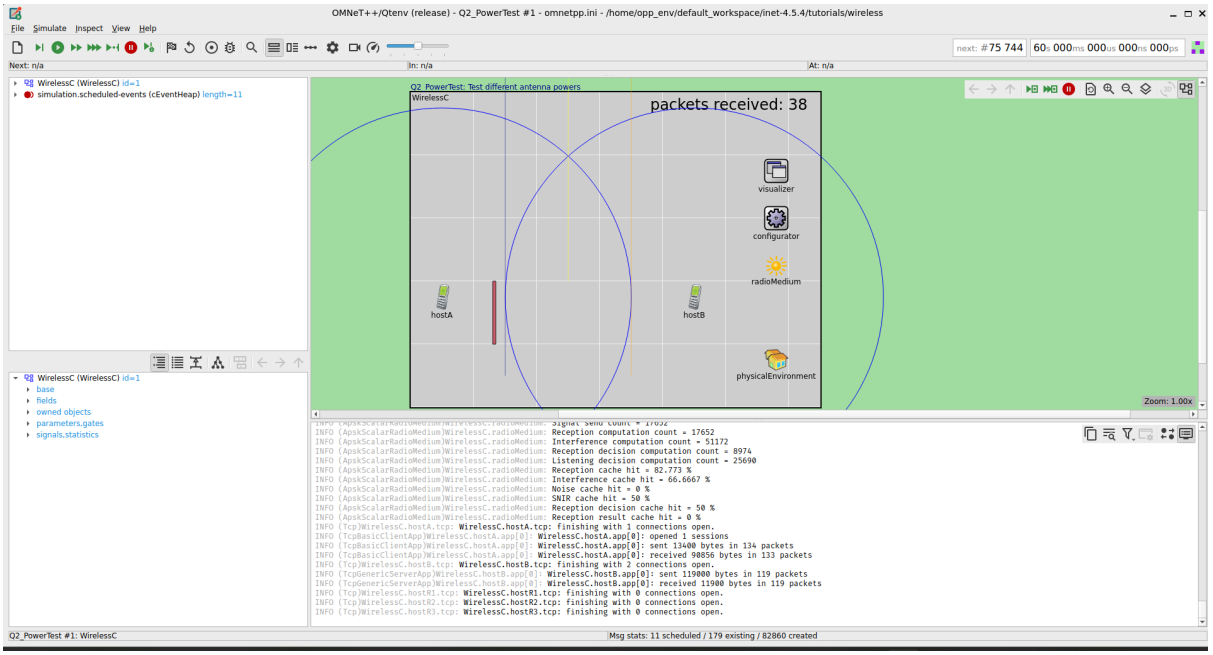


chanelper 0.2 windowsize 1

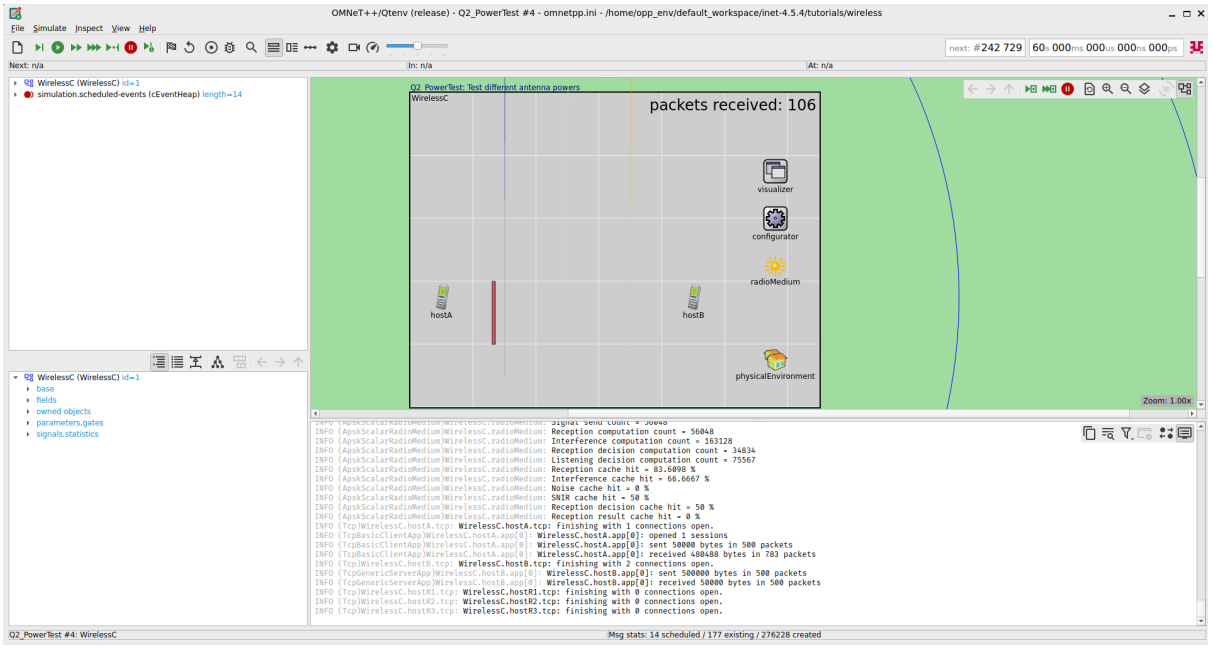
Screenshots for Q2



1mW simulation



2mW simulation



15mW simulation